

Block size: 64 - 128 - 256 - 512

↳ Only small differences; 512 was slightly faster

⇒ enough threads were available at every size to saturate memory

Array Size: 2^{10} - 2^{26}

↳ small: ~6 GB/s; large: ~249 GB/s

⇒ small arrays are dominated by launch overhead; large arrays expose memory bandwidth

Memory access: contiguous vs. stride 4

↳ stride 4 was ~5.5x slower

⇒ neighboring threads should access neighboring memory

```
Contiguous:
Threads: 0 1 2 3 4
Memory:  0 1 2 3 4    → packed/coalesced transactions

Stride 4:
Threads: 0 1 2 3 4
Memory:  0 4 8 12 16  → scattered, more transactions
```

Data Type: FP32 vs. FP16

↳ FP16 took half the time, but both reached ~249 GB/s

⇒ FP16 moves half as many bytes; confirms memory-bound behavior

Element/thread: 1 vs 4

↳ 4 elements/thread was ~2% slower

⇒ fewer threads did not reduce the main cost: memory traffic

float4: scalar vs. vectorized access

↳ float4 was slightly slower

⇒ wider instructions do not necessarily help on already bandwidth-bound kernel

grid-stride: full grid vs. reusable fixed grid

↳ slightly slower than basic

⇒ more flexible for large workloads, but has loop/index overhead

Main conclusion

Vector addition performs very little computation but transfers lots of data. Performance therefore depends mainly on memory bandwidth and coalesced access—not on reducing arithmetic instructions or giving each thread more work.

“why does increasing the block size stop helping?”

⇒ Even the smaller block sizes provide enough active warps to hide memory latency and saturate memory bandwidth

Once bandwidth is saturated, larger block cannot improve

“Is this kernel limited w/ computation or memory b.width?”

≠ we do a little computation and a lot memory transfer;
so memory-bandwidth

“How close is it to GPU's theoretical memory b.width?”

⇒ 24 GPU's bandwidth is 300 GB/s

and we hit around ~250 GB/s mostly } 83%

"Why are small arrays inefficient?"

⇒ Small arrays create too few blocks to use the whole CPU. Many GPU resources stay idle

"What happens when memory access is not contiguous?"

⇒ Effective bandwidth decreases

on stride-4 it performs more memory transactions and transferring more hardware data per element

strided access reduces memory coalescing.

*merge, unified whole